

Day 4 — System Design & Integration Plan

Date: 2 July 2026

1. What Today Was About

Today was a planning, design and integration day — not a hardware day. No new components were wired. Instead the team stepped back, reviewed every project objective honestly, reorganised into clear tracks, defined a single shared data format, and built the first working software: two dashboards. This document brings all of that together in one place and lays out how we plan to build, integrate and test the full system in real life.

The one companion document to read alongside this is the parts list (Parts_List_All_Phases.docx), which has the full breakdown of what we plan to buy across all phases.

2. Project Objectives — Honest Status

The original objectives and where each one truly stands today:

#	Objective	Status	Notes
—	Backend server + API (all data flows here)	Not started (planned)	Track C — assigned, starting now
I	GPS live location	Hardware done	NEO-6M working, real fix in Chennai (6–7 sats). Not yet sent to backend
II	Camera + mic (lane / drowsiness)	Parked	Needs Raspberry Pi / phone-class vision hardware, not ESP32 — later phase
III	Real-time OBD data (RPM, load, speed, etc.)	Partial	CAN proven by loopback. ELM327 hands-on pending. 32-byte payload designed, not car-validated
IV	Tyre parameters as 16 bytes	Designed only	16-byte packet defined. No TPMS sensor tested yet

3. How We Have Planned It — The Reframe

A few original goals could not be done the way first imagined, so today we reframed the project around what is realistically buildable. The key decisions:

3.1 Vehicle control moves to our own built platform

“Control a vehicle over CAN” will be shown on a small car we build ourselves, not a real car. An ELM327/OBD-II adapter can only read a real car’s ECU — it cannot send throttle or brake commands (manufacturers block that for safety). So we build our own platform where the ESP32 sends commands over CAN (via MCP2515) to a motor node. This is safe to test and demonstrates CAN more deeply than reading alone.

3.2 Three parallel tracks

- Track A — Real-car sensing: ELM327 + TPMS, packed into the 32-byte and 16-byte payloads.
- Track B — Autonomous platform we build: drive ~100 m forward, controlled braking, collision avoidance.
- Track C — Backend + API: receives all data, stores it, serves both dashboards.

3.3 Objective II (camera) parked

Lane and drowsiness detection is computer vision — it needs a Raspberry Pi or phone, not an ESP32. It is noted honestly as a later phase, not pretended to be in progress.

3.4 WebSocket chosen over MQTT (for now)

Both dashboards are browsers, which speak WebSocket natively, and we have only a few devices, so WebSocket is less to deploy. We can revisit MQTT later for many vehicles or a cellular link — the message format is transport-agnostic, so it is not a one-way decision.

3.5 Collision-avoidance requirement

A core requirement for the built vehicle is that it must automatically sense obstacles in its path while moving and avoid them — slowing and stopping (and, as a later goal, steering around them). This sits on the Track B platform and shapes both the sensor choice and the control logic.

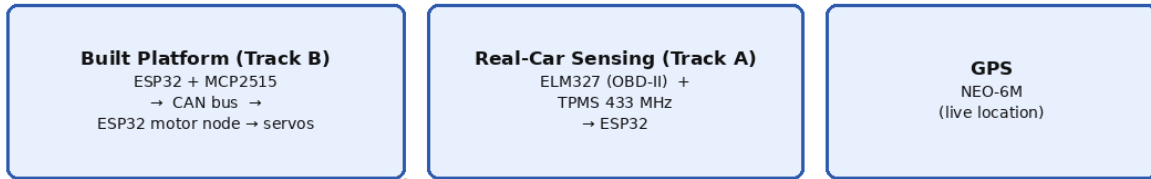
4. System Architecture

The system has three layers: the hardware (ESP32 boards with GPS, CAN and sensors, plus the platform we build), a backend (WebSocket relay + storage + API), and two browser dashboards on top.

Data moves in two formats. On the hardware link it travels as compact binary — the 32-byte vehicle packet and 16-byte tyre packet. It is unpacked into JSON just once, at the ESP32/backend boundary, and only JSON crosses the WebSocket to the browsers. This means nobody working on a dashboard ever touches raw bytes.

System Architecture

HARDWARE



BACKEND (planned)



DASHBOARDS



compact binary — 32-byte vehicle + 16-byte tyre packets

JSON over WebSocket

commands (JSON → CAN)

Figure: System architecture — hardware → backend → two dashboards (binary on the hardware link, JSON over WebSocket; commands flow back down to the platform)

5. The Two Dashboards

Two separate dashboards were designed and built today, sharing one visual style so they read as one product. Both are built against the shared message schema (Section 6).

5.1 Dashboard 1 — Telemetry Monitor

Read-only monitor for real-car data (Objectives I, III, IV). It shows drivetrain and dynamics (speed, RPM, gear, clutch, brake, fuel), TPMS and cabin (4 tyres, AC), engine health (coolant, intake, throttle, load, battery, DTC), and GPS with a live map and breadcrumb trail. Per-panel freshness dots grey out stale data.

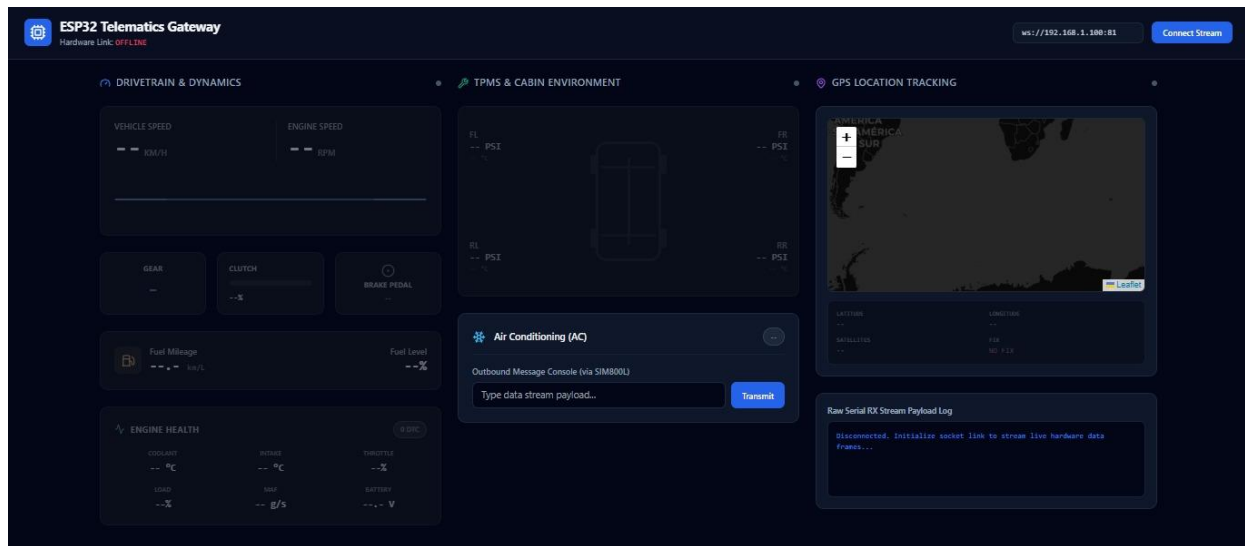


Figure: Dashboard 1 — Telemetry Monitor (deployed at lus.nalusa.space)

Deployed: lus.nalusa.space

5.2 Dashboard 2 — Vehicle Control Console

Two-way console for the platform we build (Track B). The left half sends commands — Forward, Stop, Brake, a target-speed slider, and a large Emergency Stop. The right half shows live feedback — distance travelled toward the 100 m target, current speed, obstacle distance, drive state, battery and heading, plus a collision-avoidance banner (CLEAR / SLOWING / BRAKING) and command/status logs.

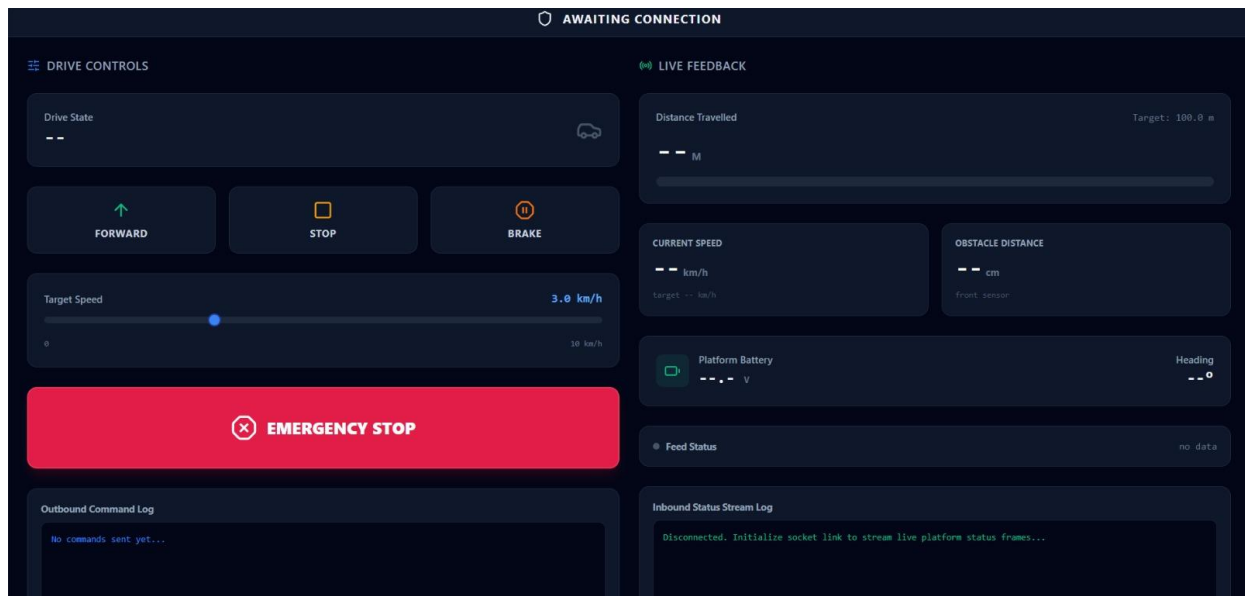


Figure: Dashboard 2 — Vehicle Control Console (deployed at dashboard2.nalusa.space)

Deployed: dashboard2.nalusa.space

Source code (both dashboards): github.com/Shaahirahamed/Car-Dashboard-Let-Us-Support

6. Shared Message Schema

Both dashboards and the backend use one identical JSON format so nothing drifts. There are three message types, each carrying a type and a millisecond timestamp (ts):

- telemetry — hardware → Dashboard 1: vehicle, tyres and gps sub-objects.
- platform_status — platform → Dashboard 2: drive_state, avoidance_state, distance_m, speed_kmh, obstacle_cm, heading, battery.
- command — Dashboard 2 → platform: forward, set_speed, stop, brake, estop.

The ts field drives the freshness indicators (a value greys out if it has not updated within 3 seconds). A safety rule is built in: an emergency stop always wins and the platform stays stopped until a fresh forward command is sent.

7. How We Plan to Build, Integrate and Test It

We do not wait for all hardware before building. The plan is designed so software and hardware progress in parallel and meet cleanly.

1. Fake-data first: a small script emits schema-shaped JSON so the backend and both dashboards can be fully built and tested with no hardware present. Dashboard 2 also has a local simulate mode for the same reason.
2. Backend relay: stand up the WebSocket relay + storage + API. Point both dashboards at it instead of directly at the ESP32.
3. Real telemetry (Track A): connect the ELM327 + TPMS on a real car, pack the 32-byte and 16-byte payloads, and confirm they render correctly on Dashboard 1.
4. Build the Phase-1 CAN car (Track B): assemble the servo platform with the two-node CAN bus (ESP32 + MCP2515 → CAN → second ESP32 + MCP2515 → servos).
5. CAN control test: send a command from Dashboard 2 and confirm it crosses the CAN bus and moves the car — the core demonstration.
6. Distance + avoidance: add the wheel encoder (with GPS cross-check) for the ~100 m estimate, and the ultrasonic sensor for detect-and-stop collision avoidance.
7. Full run: demonstrate the ~100 m autonomous drive with collision avoidance, all commanded over CAN and shown live on Dashboard 2.

8. Parts & Procurement

The full parts breakdown — every phase and every component — is in the companion document `Parts_List_All_Phases.docx`. In summary:

- Phase 1 (cheap CAN-controlled proof car, servos): about ₹2,000 in new parts, reusing the ESP32 boards, CAN modules and GPS we already own.
- Later phases (full encoder-motor car, autonomy sensors, and real-car ELM327 + TPMS sensing): roughly ₹14,000 more, to be ordered after Phase 1 is proven.

Recommended approach: approve and run Phase 1 first to prove the CAN control and autonomy cheaply, then approve the later phases. See the parts document for the itemised list, options and prices.

9. Team Roles

Person	Role	Responsibility
Shahid Mihransha	Team Lead	Overall planning and sequencing of the project; hardware bring-up and real-world GPS testing (obtained the live fix in Chennai); backend and parts-research support across the tracks; and coordination of the team's deliverables.
Sapthagiri	Dashboard 1 — Telemetry	Built and extended the Telemetry Monitor — GPS trail, freshness indicators, engine-health panel, bound to the shared schema.
Sathish	Dashboard 2 — Control Console	Built the Vehicle Control Console — drive/stop/brake/e-stop controls, target-speed slider, and live platform feedback.
Shaahir	Backend + API	Backend server — WebSocket relay, storage, and REST history endpoints (starting now).
Venkat	Platform hardware / procurement	Platform build and parts research; sourcing for the Phase-1 car (no buying until budget sign-off).
Pavan	Real-car sensing + documentation	Early-stage: basic ELM327/TPMS sensor and algorithm research; project documentation.

10. Next Steps

- Secure sign-off on the reframe and the Phase-1 parts budget (~₹2,000).
- Order Phase-1 parts; build the servo car and two-node CAN wiring.
- Build the backend + fake-data sender so the software pipe is testable now.
- Do the ELM327 + TPMS hands-on on a real car; finish the payload packing.
- Demonstrate the Phase-1 ~100 m CAN-controlled run with collision avoidance, then propose the later phases.